

demo_attack_profile.py — Explained Simply

A plain-English, part-by-part walkthrough of `test/C4/demo_attack_profile.py` (WebSentinel — C4: Browser Artifact Forensics)

What this file is, in one sentence: it is a test script that builds a **fake but very realistic crime scene** on disk (real Chrome database files, planted with a hacking story), then runs your **actual, real C4 pipeline** over it to prove every part of the system correctly detects the attack.

Analogy: *imagine training a sniffer dog. Instead of testing the dog on one small object at a time, you build a whole fake crime scene — a house with a break-in, a stolen item, footprints, a getaway car smell — and then let the dog work the entire scene from start to finish. If the dog finds everything, you know the dog (your C4 system) really works in the real world, not just in a lab test.*

1. The introduction comment

lines 1–21

The top of the file is just a comment (text that doesn't run as code) explaining the plan. It says there is another file, `test_units.py`, that tests each small function separately. This file is different — it tests the **whole chain** together:

```
run_extraction -> apply_single_artifact_rules -> run_correlation
                -> run_mitre_mapping -> reporter -> service verdict
```

In simple words, the data flows through six stages, like a factory assembly line:

1. **Extraction** — read raw data out of browser files
2. **Rules** — flag obviously suspicious single events
3. **Correlation** — connect the dots between different events
4. **MITRE mapping** — label findings with official attack-technique names
5. **Reporter** — turn results into HTML / JSON / SIEM reports

6. **Service verdict** — give one final risk score (e.g. "HIGH")

This script checks that all six stages fire correctly on one planted case.

2. Setup code

lines 22–36

```
import os
import sys
sys.path.insert(0, ...)
from core.c4 import demo_case, reporter, service
from core.c4.extractor import sha256
```

This part just prepares the script to run:

- `sys.path.insert(...)` tells Python where to find your project's code, so the imports below work no matter where the script is run from.
- It imports three files from your C4 module: `demo_case` (builds the fake evidence), `reporter` (makes reports), and `service` (gives the final verdict).
- It also imports `sha256`, a function used to "fingerprint" a file so you can prove it wasn't changed.

Then a few lines set up colored text (green for PASS, red for FAIL) so the terminal output is easy to read.

3. The three helper functions

lines 38–59

These are small reusable tools used everywhere else in the file:

```
check(name, condition, detail)
```

This is the heart of the whole script. Every single test in this file is one call to `check(...)`. You give it:

- **name** — a description of what is being tested, e.g. "History rows survive the round trip intact"
- **condition** — something that is either True or False
- **detail** — optional extra info to print, like numbers or examples

It prints `PASS` or `FAIL`, saves the result in a list called `results`, and returns whether it passed.

`section(title)`

Just prints a big heading like `== Stage 1 ==` to visually separate parts of the output. Purely cosmetic — makes the terminal readable.

`types_in(events)`

Counts how many events of each artifact type came out of the pipeline (e.g. 40 history entries, 5 cookies, 3 downloads). It returns something like `{"history": 40, "cookie": 5, "download": 3}`. This count is used later to check nothing went missing.

4. Planting the evidence and running the real pipeline

lines 62–88

This is where the fake crime scene gets built.

```
case = demo_case.build_case_profile()
```

This calls a separate file, `demo_case.py`, which physically writes **real Chrome-format SQLite database files** to disk — a real `History` file, a real `Cookies` file, a real `Login Data` file — and fills them with a made-up but coherent attack story: some normal browsing, a burst of suspicious activity, a breach, a malicious download, and stolen-looking credentials.

*This is **not** fake data sitting in memory — it is genuine files on your hard disk, in exactly the format a real Chrome browser would produce. That's important: it proves your extractor works on real file formats, not on shortcuts.*

```
hash_before = sha256(os.path.join(profile, "History"))
```

Before running anything, the script takes a "fingerprint" (hash) of the History file. Later, it checks this fingerprint hasn't changed — proving the analysis never modified the original evidence (very important in real forensics).

```
result = demo_case.run_case_pipeline(case["root"])
```

This is the big moment: it runs your **actual, unmodified, production C4 pipeline** — the same code that would run on a real investigation — over the planted files.

Two checks follow, confirming the fake evidence itself was built correctly (files exist on disk, and the fake saved passwords are properly encrypted the same way real Chrome encrypts them — with DPAPI-wrapped AES-256-GCM).

5. Stage 1 — Extraction checks

lines 91–143

This section proves your **extractor** can correctly read every type of evidence back off the disk files. Each item below is one `check(...)` call:

What is checked	In simple words
All 7 artifact types found	history, cookie, download, credential, extension, session, and localStorage data all came back — nothing silently skipped
Extensions found in both places	Chrome stores extension info in two different spots on disk; the extractor checks both
Side-loaded extension flagged	An extension installed outside the official Chrome Web Store gets correctly marked as suspicious
Session tabs recovered	Open browser tabs (from "restore session" files) are correctly read, including old and new text encodings
History count matches exactly	The number of visits read back equals exactly what was planted — nothing lost or duplicated
Evidence manifest hashes every file	Every source database gets its own fingerprint recorded, for integrity proof
Downloaded file hash is real	The hash of a dropped file is actually computed from real file bytes, not just made up
Passwords decrypted for real	Saved passwords go through the real Windows DPAPI + AES-GCM decryption chain, same as a real investigation would need
No plaintext password leaks	Even though passwords are decrypted internally, the actual plaintext password text never appears in any report — only a masked/partial version

6. Stage 2 — Rule engine checks (R01–R06)

lines 146–170

Your C4 module has six built-in "single-artifact" rules — simple red-flag checks that look at **one event at a time** (not connecting different events together yet). This section proves all six rules correctly fire on the planted case:

Rule	What it catches
R01	A suspicious domain (e.g. fetching raw text from pastebin.com)
R02	A dangerous file download (e.g. a <code>.exe</code> file)
R02b	Chrome's own built-in "this file looks dangerous" flag
R03	A sensitive session cookie (e.g. a login/session token)
R04	A saved credential (username/password) record
R05	An extension asking for risky permissions
R06	A "URL burst" — 25 URLs visited in under 60 seconds (abnormal speed)

The code loops through every event, collects which rules fired on it, and counts them:

```
fired = {}
for e in events:
    for f in e.get("rule_flags", []):
        fired.setdefault(f["rule"], 0)
        fired[f["rule"]] += 1
```

Then for each rule it checks the count is greater than zero — meaning the rule actually caught something in the planted case.

The last check in this section is just as important the other way round:

"Ordinary browsing stays unflagged" — this proves the system is *not* flagging everything as suspicious (a "blanket flag" bug). At least 80% of the normal baseline browsing must remain clean/unflagged. A detector that flags everything is useless — this check guards against that.

7. Stage 3 — The seven cross-table detectors (A–G)

lines 173–232

This is the most advanced part of C4: the **correlation engine**. Instead of looking at one event at a time (like Stage 2), it connects **different types of evidence together** to spot patterns a single rule could never catch alone.

A. Co-occurrence detector

Checks that on the "breach domain," at least 4 different artifact types (e.g. history + cookie + download + credential) all show up together — a strong sign that domain is the center of the attack.

B. Orphan detection (domain-based)

Finds artifacts (like a cookie or download) tied to a domain that was **never actually visited** in history. That's suspicious — it can mean history was deliberately cleared, or the artifact appeared some other way.

B2. Orphan detection (session-based)

A special case: an open browser tab exists for a domain, but there's no history record of ever navigating there. This can mean history was wiped while the tab stayed open, or the page never fully loaded.

C. Temporal anomaly detector

Compares activity timing against the user's own normal habits. The fake user normally browses 9 AM–5 PM. This detector proves that activity happening at 3 AM — a time this user never normally browses — gets flagged as abnormal.

D. Attack chain detector

Looks for a specific dangerous **order** of events: browse to a site → download something → a credential shows up. That exact sequence, in that exact order, is a classic attack pattern (e.g. visiting a phishing site, downloading malware, then credentials being stolen/used).

E. Domain risk clustering

Ranks all domains by how suspicious they are overall (combining event count and variety of artifact types). Checks that the planted "breach domain" comes out ranked **#1** — the riskiest

domain in the whole case.

F. Credential reuse detector

Checks that the same username/identity was reused across at least 3 different domains — a common bad security habit (and a classic sign of credential-stuffing risk if one of those sites gets breached).

G. Download → exfiltration detector

Looks for the pattern: a file gets downloaded from one domain, and then — within 2 minutes — the browser navigates out to another (exfiltration) domain. This can indicate data being copied out after a compromise.

Final check in this section: "All seven detectors produced findings on one case" — proves that on this single planted scenario, every one of the seven detectors (A through G) found at least one real result. None of them stayed silent.

8. Stage 4 — MITRE ATT&CK mapping

lines 235–253

MITRE ATT&CK is a well-known, industry-standard catalogue of attacker techniques (used by real security teams worldwide). Instead of just saying "this looks suspicious," C4 labels each finding with an official technique ID (like a police case-law reference) so analysts instantly know what kind of attack pattern it resembles.

This section checks three things:

- Every single correlation finding from Stage 3 got matched to a real MITRE technique ID — nothing was left unlabeled.
- Findings are graded by severity (High / Medium / Low), and at least one High severity finding exists — the planted breach is serious enough to be marked High.
- The case touches at least 3 different MITRE "tactics" (broad attack categories, e.g. Credential Access, Exfiltration, Execution) — proving this is a multi-stage attack, not a single narrow issue.

9. Stage 5 — Reporting and final verdict

lines 256–285

This section proves the results can actually be delivered to a human analyst or another system.

Check	In simple words
HTML report renders the case	A human-readable HTML report is generated and actually contains the breach domain and a MITRE findings table
SIEM export works	A machine-readable export is generated, in the format a real Security Operations Center (SOC) tool would ingest
Files written to disk	JSON, HTML, and SIEM report files are actually saved as real files, not just built in memory
Risk verdict escalates	The final one-line verdict (via <code>service.get_summary</code>) correctly comes out as HIGH or CRITICAL — the system knows this is a serious case
Evidence stayed unchanged	The History file's fingerprint (hash) from the very start of the script is compared again here — proving the entire analysis never modified the original evidence, which is a core forensic requirement

10. Stage 6 — Function coverage

lines 288–306

This last technical section isn't testing correctness — it's proving **completeness**. It calls a few more read-only functions (like `service.get_default_profile_path()`) just so they get exercised too, then asks `demo_case.coverage_report()` how many of the C4 module's tracked functions were actually called somewhere during this run.

Think of it like a maintenance checklist for a car — this doesn't check if the engine runs well (that was already tested above), it just confirms that every single part on the checklist was actually touched and inspected at least once during the test drive, none skipped.

If any function was never called, its name is listed under "missing" so you know exactly what still needs to be exercised.

11. The final summary

lines 309–318

```
passed = sum(1 for _, p in results if p)
total = len(results)
...
sys.exit(0 if passed == total else 1)
```

The script counts how many of all the `check(...)` calls passed, prints a final scoreboard like "24/24 passed," and exits with code `0` (success) only if absolutely everything passed. If even one check failed, it exits with code `1` (failure) — which is important because it lets other tools (like a CI pipeline or the dashboard's Live Test Runner) automatically know whether the whole demo succeeded or not.

Big picture takeaway: this single script is a complete, live proof that your C4 module works correctly on real evidence, from raw disk files all the way to a final "HIGH risk" verdict — covering extraction, all 6 single-event rules, all 7 cross-table correlation detectors, MITRE ATT&CK labeling, report generation, and evidence integrity, in one run.